

App Development Project Report

App Dev Project Report

1. Student Details

Name: Anijesh Kumar

Roll Number: 23f2001075

Email: 23f2001075@ds.study.iitm.ac.in

About Me: I am a dual-degree student pursuing **Data Science & Applications** at IIT Madras and **Artificial Intelligence & Machine Learning** at Sir M. Visvesvaraya Institute of Technology. I have a strong interest in software development, backend engineering, and database-driven applications. I enjoy learning new technologies, building end-to-end systems from scratch, and continuously improving my problem-solving and coding skills through hands-on projects and real-world applications.

2. Project Details

Placement Portal Application (PPA)

Problem Statement:

To design and develop a web-based Placement Portal that enables seamless interaction between Admin, Companies, and Students. The system should support role-based access, job postings, application tracking, interview scheduling, automated email reminders, and monthly report generation for placement statistics.

Approach:

The application was built using Flask with SQLAlchemy ORM for database operations, Redis for caching, and Celery for background job processing. A three-tier architecture was implemented with Vue.js frontend, Flask Restful API backend, and SQLite database. JWT-based authentication was used for secure role-based access. Celery Beat scheduler was configured for automated daily reminders and monthly reports via SMTP.

3. AI/LLM Declaration

I used ChatGPT (GPT-5) to assist with writing parts of my backend code, including Celery task configuration, automated email remainder logic, and interview scheduling workflow. For the frontend, I received guidance on Vue.js component structure, routing configuration, and API integration with Axios. I also used AI assistance for improving code structure, variable naming, and generating minor suggestions for API endpoint design.

The overall usage of AI/LLM was approximately **15–20%**, limited only to guidance and explanation.

All core implementation, end-to-end logic, debugging, and feature integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework; RESTful API and routing
Flask-RESTful	Build RESTful API with resources and methods
Vue.js	Frontend framework for Single Page Application (SPA)
SQLite	Lightweight relational database for placement records
SQLAlchemy ORM	Defines database models and manages object–relational mapping
Werkzeug Security (pbkdf2:sha256)	Secure password hashing for admin, company, and accounts
Flask-JWT-Extended	Secure JWT-based authentication and token management
Flask-SQLAlchemy	Integration of SQLAlchemy with Flask
Flask-CORS	Enable Cross-Origin Resource Sharing between frontend and backend
Celery	Distributed task queue for asynchronous job processing
Redis	Message broker for Celery; caching layer for improved performance
Flask-Caching	Caching middleware for optimized API responses
MailHog	Local SMTP server for email testing and verification
Vite	Frontend build tool for fast development and optimized production builds
Axios	HTTP client library for frontend API requests

5. Database Schema / ER Diagram

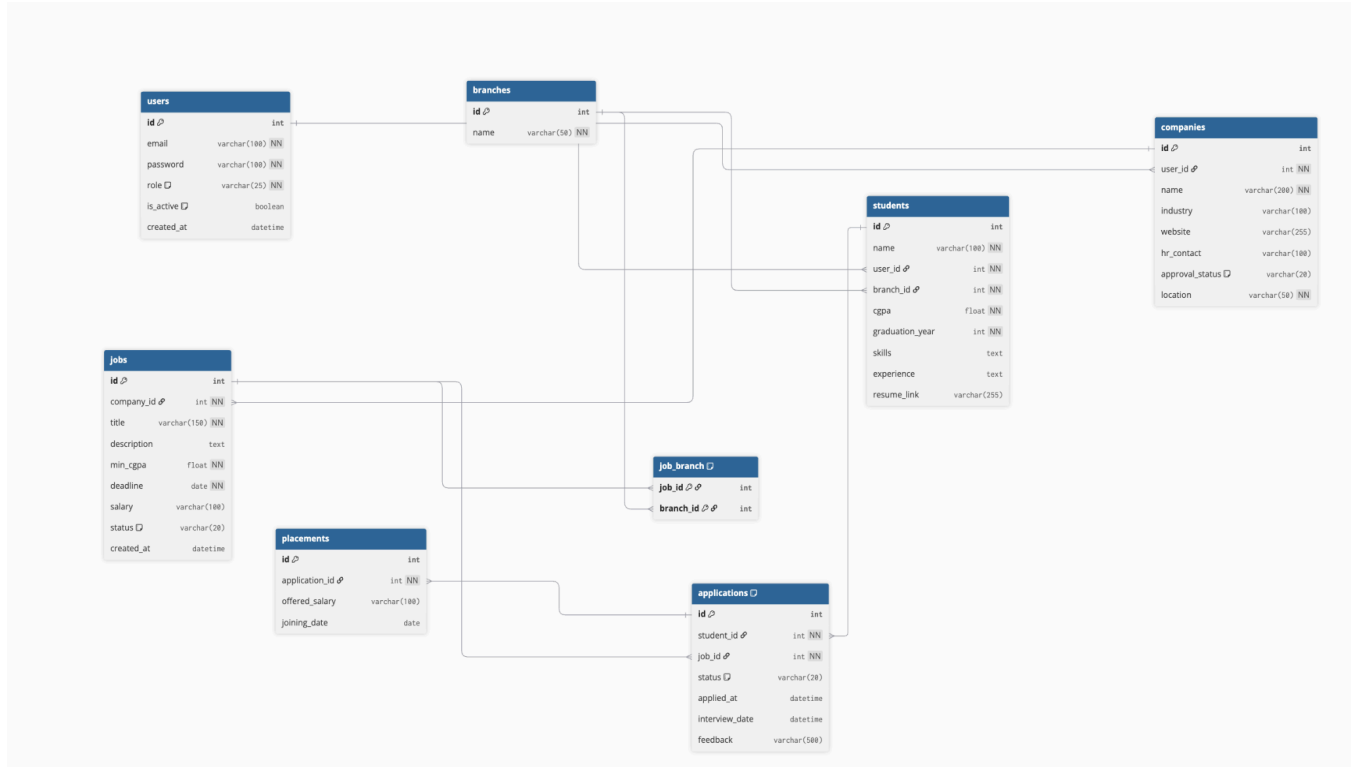
Tables:

1. **Users**— stores user accounts (id, email, password, role, is_active, created_at)
2. **Students** — stores student details (id, name, user_id, branch_id, cgpa, graduation_year, skills, experience, resume_link)
3. **Companies** — stores company information (id, user_id, name, industry, website, hr_contact, approval_status, location)
4. **Branches** — stores academic branches (id, name)
5. **Jobs** — stores placement drives (id, company_id, title, description, min_cgpa, deadline, salary, status, created_at)
6. **Job_Branch** — junction table for job-branch eligibility (job_id, branch_id)
7. **Applications** — stores job applications (id, student_id, job_id, status, applied_at, interview_date, feedback)
8. **Placements** — stores final placements (id, application_id, offered_salary, joining_date)

Relationships:

Relationship Type	From Table	To Table	Description
One-to-One	Users	Students	One user has one student profile
One-to-One	Users	Companies	One user has one company profile
One-to-Many	Branches	Students	One branch has many students
One-to-Many	Companies	Jobs	One company posts many jobs
Many-to-Many	Jobs	Branches	Jobs eligible for multiple branches
One-to-Many	Students	Applications	One student applies to many jobs
One-to-Many	Jobs	Applications	One job receives many applications
One-to-One	Applications	Placements	One application results in one placement

ER Diagram



6. API RESOURCES ENDPOINT

Endpoint	Method	Description
/api/auth/login	POST	User login (returns JWT token)
/api/auth/register	POST	User registration for student/company
/api/auth/logout	POST	User logout
/api/branches	GET	List all branches
/api/student/profile	GET, PUT	View/update student profile
/api/student/jobs/list	GET	View all available jobs
/api/student/job/<id>/apply	POST	Apply for a job
/api/student/application/list	GET	View student's all applications with status
/api/student/placements	GET	View student's placement history.

/api/student/export/csv	GET, POST	Export student's applications as CSV
/api/company/profile	GET, PUT	View/update company profile
/api/company/create/job	POST	Post new placement drive (job)
/api/company/job/list	GET	View company's posted jobs
/api/company/job/application/<id>/list	GET	View applications for a specific job
/api/company/application/<id>/shortlist	POST	Shortlist a student application
/api/company/application/<id>/reject	POST	Reject a student application
/api/company/application/<id>/accept	POST	Accept/Select student application
/api/company/application/<id>/interview	POST	Schedule interview
/api/company/export/csv	GET, POST	Export applications as CSV
/api/admin/stats	GET	View placement statistics dashboard
/api/admin/companies	GET	List all companies
/api/admin/companies/<id>/approve	POST	Approve company registration
/api/admin/companies/<id>/reject	POST	Reject company registration
/api/admin/students	GET	List all students
/api/admin/students/<id>/deactivate	POST	Deactivate student account
POST /api/admin/students/<id>/activate	POST	Activate student account
/api/admin/job/list	GET	View all placement drives
/api/admin/job/<id>/approve	POST	Approve job posting
/api/admin/job/<id>/reject	POST	Reject job posting
/api/admin/application/list	GET	View all applications
/api/admin/placement/list	GET	View all placements

7. Video Presentation

Drive Link:

https://drive.google.com/file/d/1LuyIAFGzeFYeh06Jvloa7uz6UjGbNm_3/view